Project Report

Lunar Audio: Smart Audiobook Player

Course: Mobile Application Development

Group Members:

- Khaled Rami Omar (202307853)

Delivery Date: 25 March 2026

Table of Contents

1. [App Vision](#app-vision)

3. [User Stories (Selected Non-Trivial)](#user-stories-selected-non-trivial)

1. [UI Diagrams / Sketches](#ui-diagrams--sketches)

3. [Component diagram](#component-diagram)

5. [Conclusion](#conclusion)

7. [Final Note](#final-note)

App Vision

Lunar Audio is a premium mobile audiobook app focused on smooth long-form listening, especially for M4B/M4A files.

The app solves common audiobook pain points: poor chapter support, weak resume accuracy, and inconsistent playback across sessions/devices. It offers secure user accounts, cloud-synced progress, chapter-aware playback, and a minimalist high-contrast interface designed for distraction-free listening.

Features and Requirements Specification

This section maps the requested project requirements to the implemented Android solution.

Implemented feature overview

Lunar Audio is implemented as a Kotlin Android app using Jetpack Compose and an MVVM-style architecture. Core playback is built on Media3, where a MediaSessionService (PlaybackService) handles

background playback and system integrations. UI and service are connected through a MediaController wrapper (AudiobookPlayer) and Compose state flows.

The app includes the requested core screens:

- Player screen with play/pause, seek, chapter support, speed control, and sleep timer.

- Profile and settings flows for notification preferences and app behavior.

It also includes advanced features from the synopsis:

- Biometric lock entry flow (`LibraryLockedScreen` and `BiometricPrompt` in `MainActivity`).

- Notification services (local scheduled reminders and FCM handling).

Requirements traceability matrix

```
| ID | Requirement | Status | Notes / Evidence |

| R1 | M4B/M4A audiobook playback | Implemented | Media3 `ExoPlayer` + `PlaybackService` (`MediaSessionService`) |

| R3 | Library browsing and search | Implemented | `LibraryViewModel` + `LibraryScreen` reactive filtering |

| R5 | Resume from saved position | Implemented | Position persisted and restored in `PlayerViewModel` + repositories |

| R7 | Sleep timer | Implemented | Timed mode + end-of-chapter mode in `PlayerViewModel` |

| R9 | Firebase authentication for user account flows | Partially integrated | Auth repository is present; dedicated sign-in/sign-up UI flow is not fully wired in navigation |

| R11 | Notification workflows (daily, streak, milestones) | Implemented (with caveats) | Scheduler + trigger helper + FCM service; multiple silent-failure catches remain |
```

Non-functional requirements and engineering choices

- **Performance and UX responsiveness:** Playback state and progress are exposed as `StateFlow`, which keeps Compose UI reactive and avoids heavy polling in the UI layer.

- **Reliability:** Error handling exists in most repositories, but several notification and remote paths still swallow errors, reducing observability.

- **Compatibility:** Minimum SDK 26 and modern target/compile SDK 35 configuration.

User Stories (Selected Non-Trivial)

The following stories were selected because they combine multiple subsystems (UI, media service, persistence, and cloud/backend features).

User Story 1 - Chapter-aware listening

Story: As a listener, I want to navigate by chapter so I can jump directly to meaningful parts of long audiobooks.

Acceptance focus:

- The player can seek to chapter start.
- Chapter-relative progress is shown.

Implementation status: Implemented.

User Story 2 - Exact resume after interruption

Story: As a listener, I want to resume exactly where I stopped so I do not lose context in long sessions.

Acceptance focus:

- Re-opening a book resumes from saved position.

Implementation status: Implemented.

User Story 3 - Cross-device continuity

Story: As an authenticated user, I want my progress in the cloud so I can continue on another device.

Acceptance focus:

- Cloud progress can be pulled and merged locally.

Implementation status: Partially integrated.

User Story 4 - Secure access to private library

Story: As a user, I want biometric gatekeeping so unauthorized users cannot open my audiobook library.

Acceptance focus:

- Unlock requires successful biometric authentication.

Implementation status: Partially integrated.

- Cha
- Curr

Playe

- Pos
- Res

Progr

- Prog
- Rea

Progr

- Loc

Biom

User Story 5 - Smart engagement notifications

Story: As a listener, I want reminders and milestone notifications so I can maintain consistency and motivation.

Acceptance focus:

- Streak and milestone events can trigger notifications.

Implementation status: Implemented with operational caveats.

Diagrams

UI Diagrams / Sketches

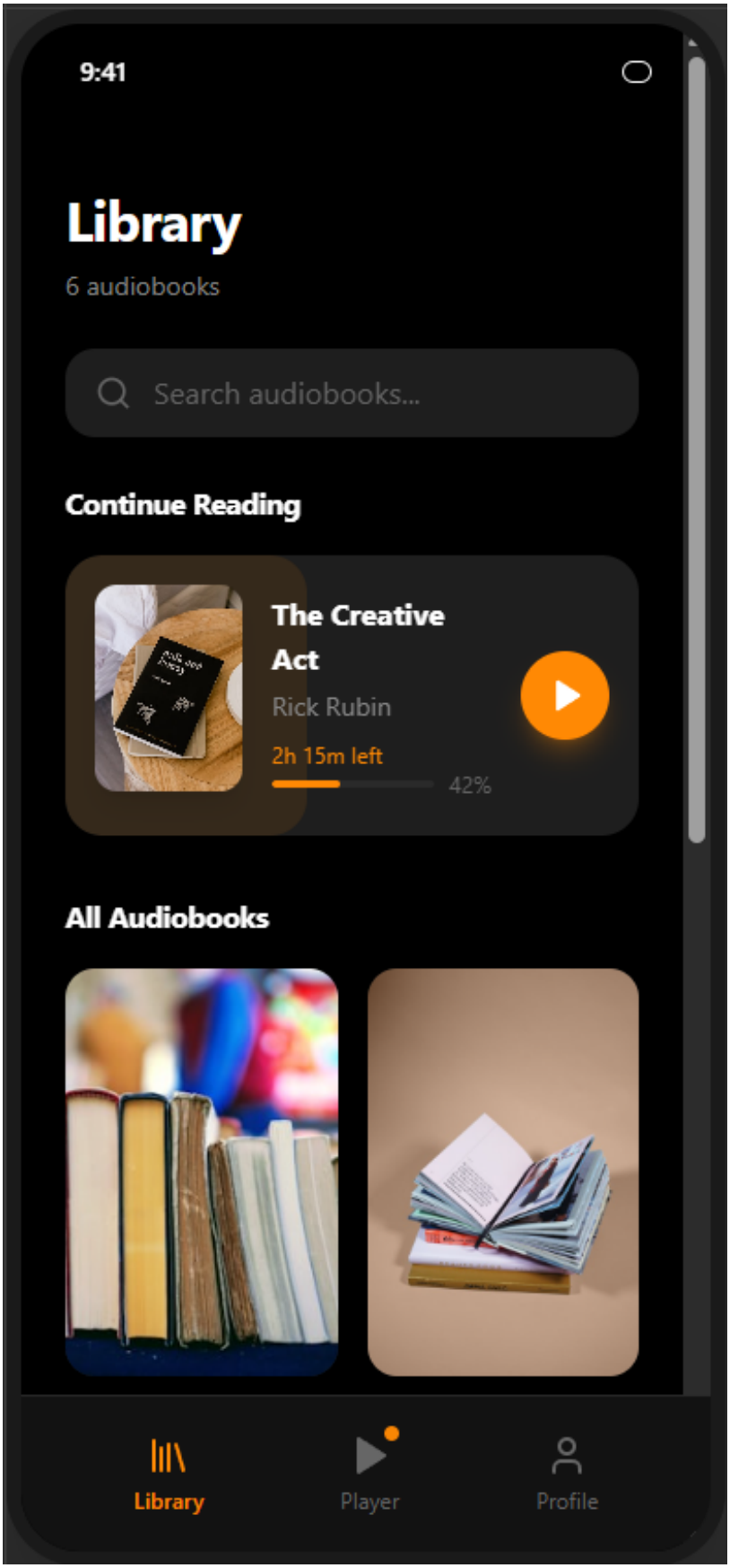
Primary UI sketches and emulator captures are available in project assets:

- Library sketch:

- Dail

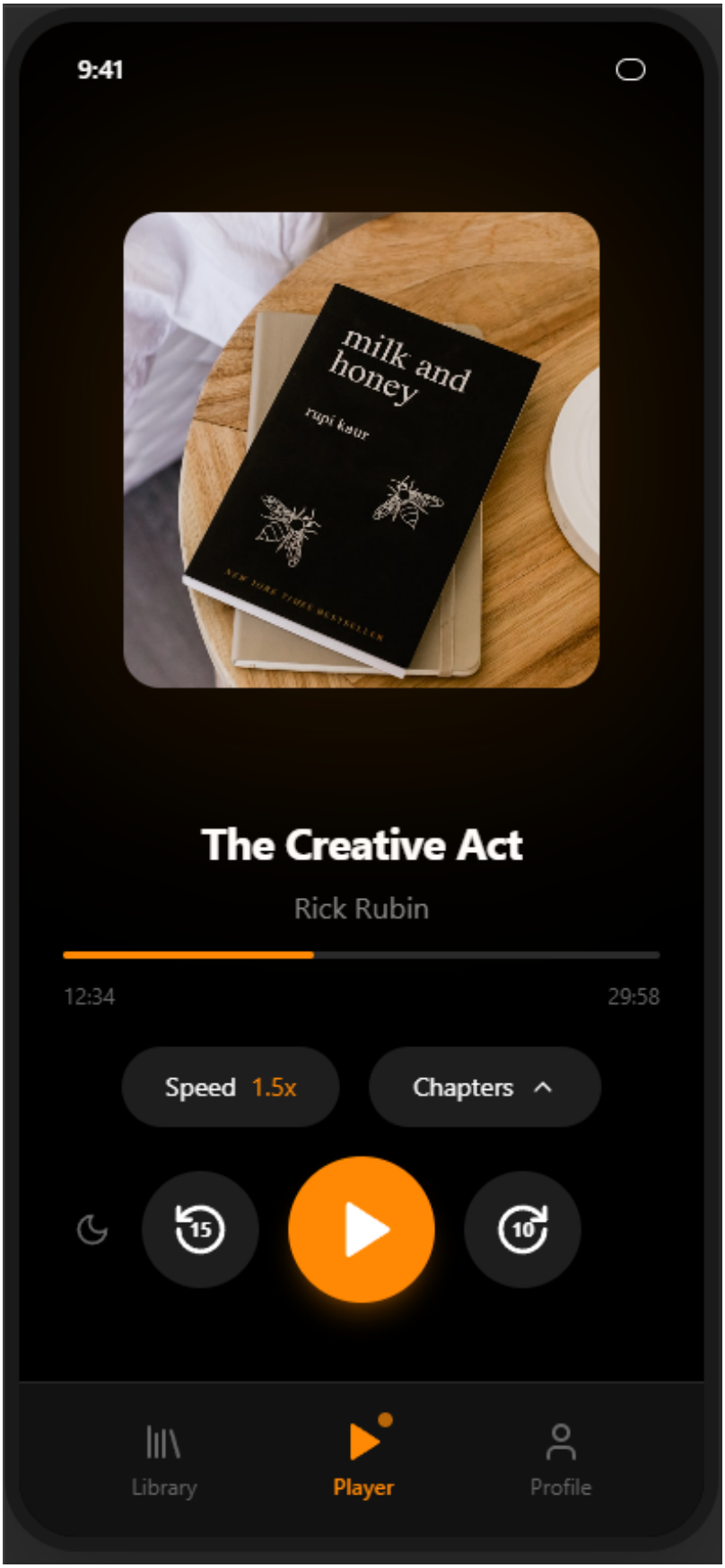
- Noti

Daily/



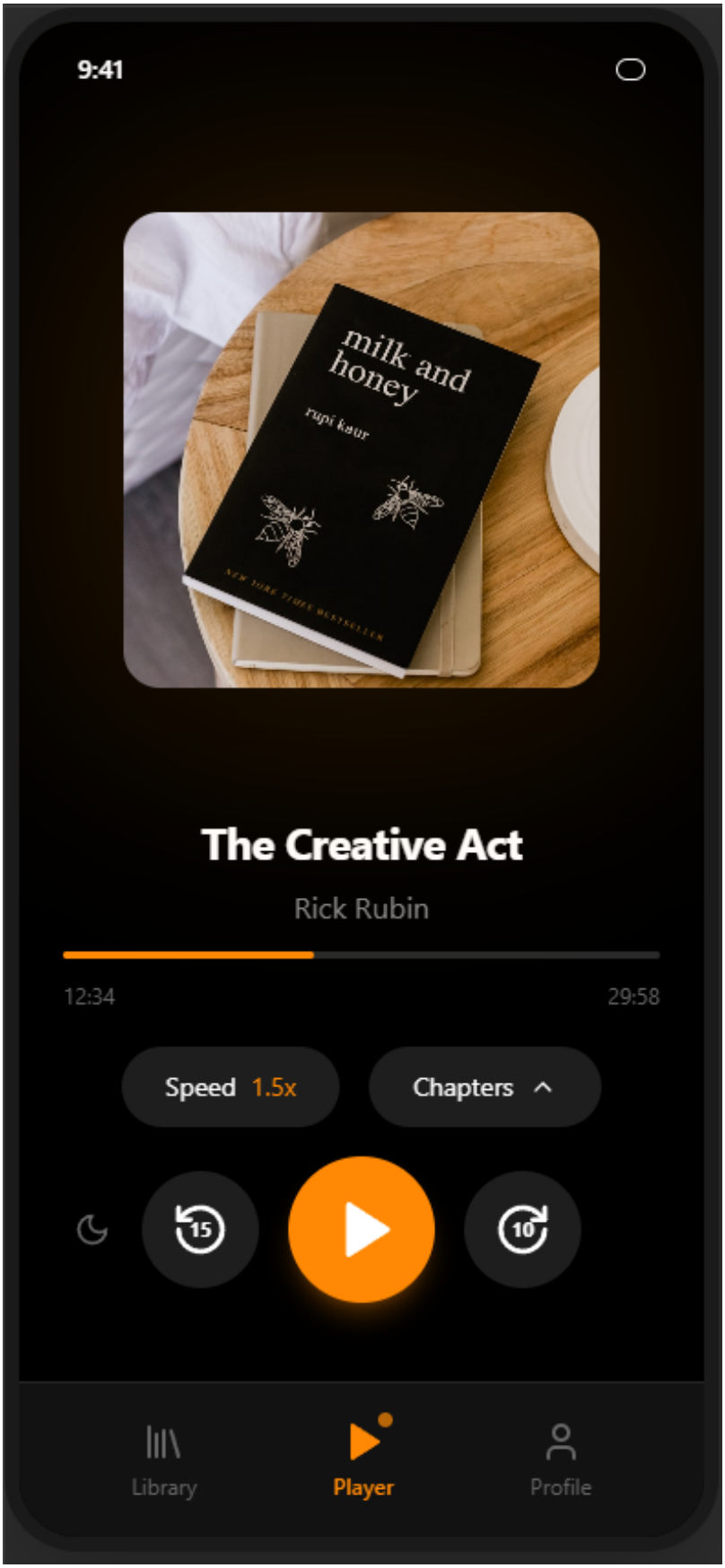
Library

- Player sketch:



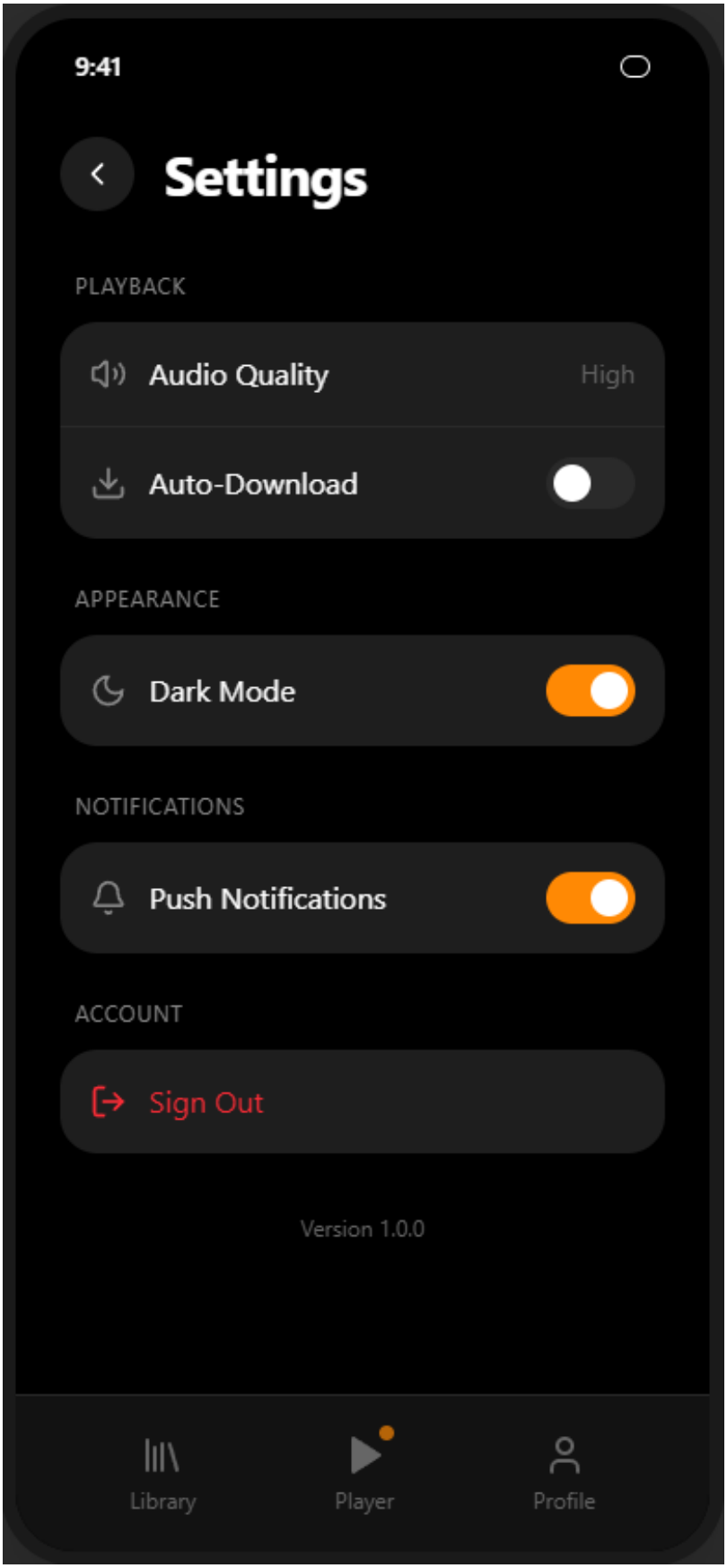
Player

- Profile sketch:



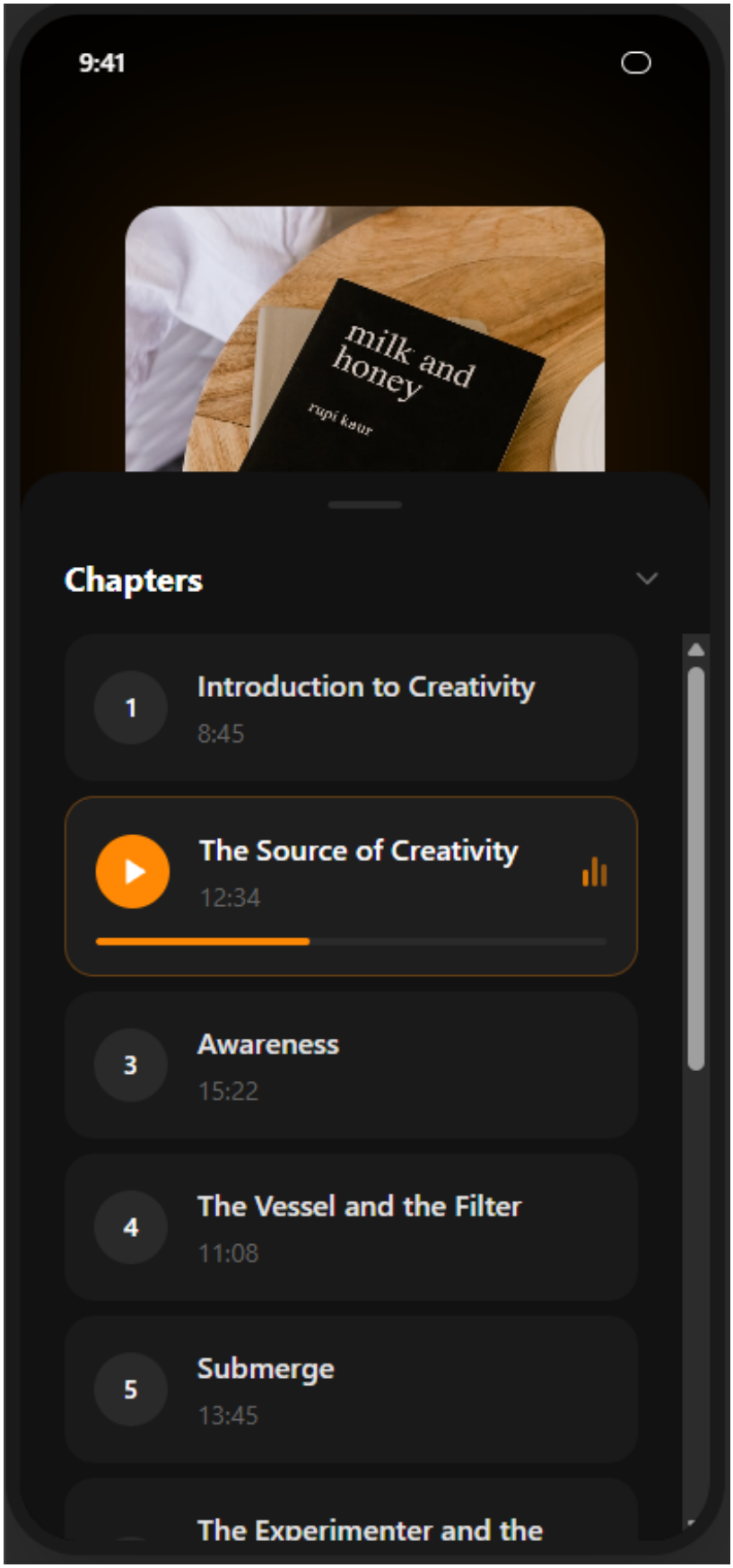
Profile

- Settings sketch:

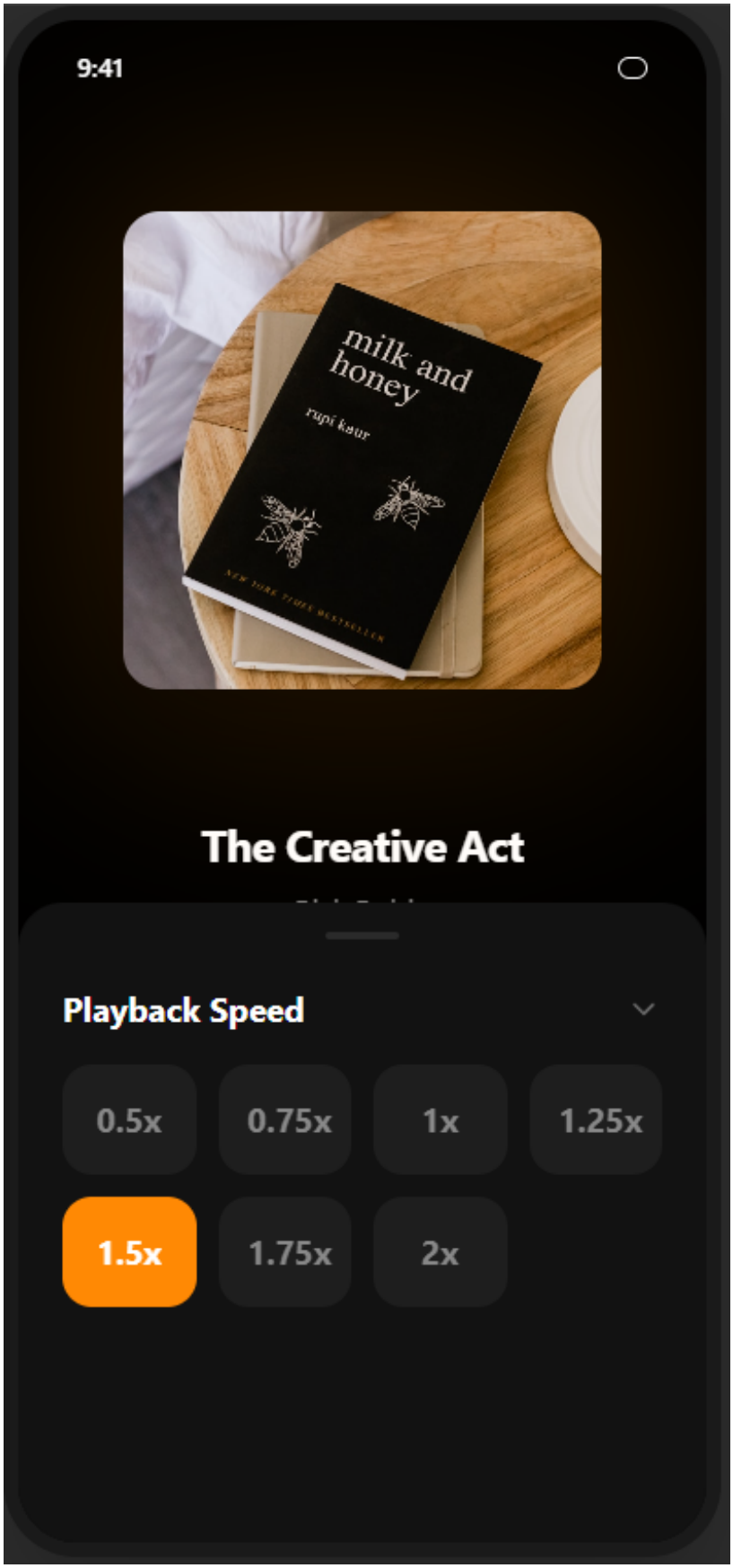


Settings

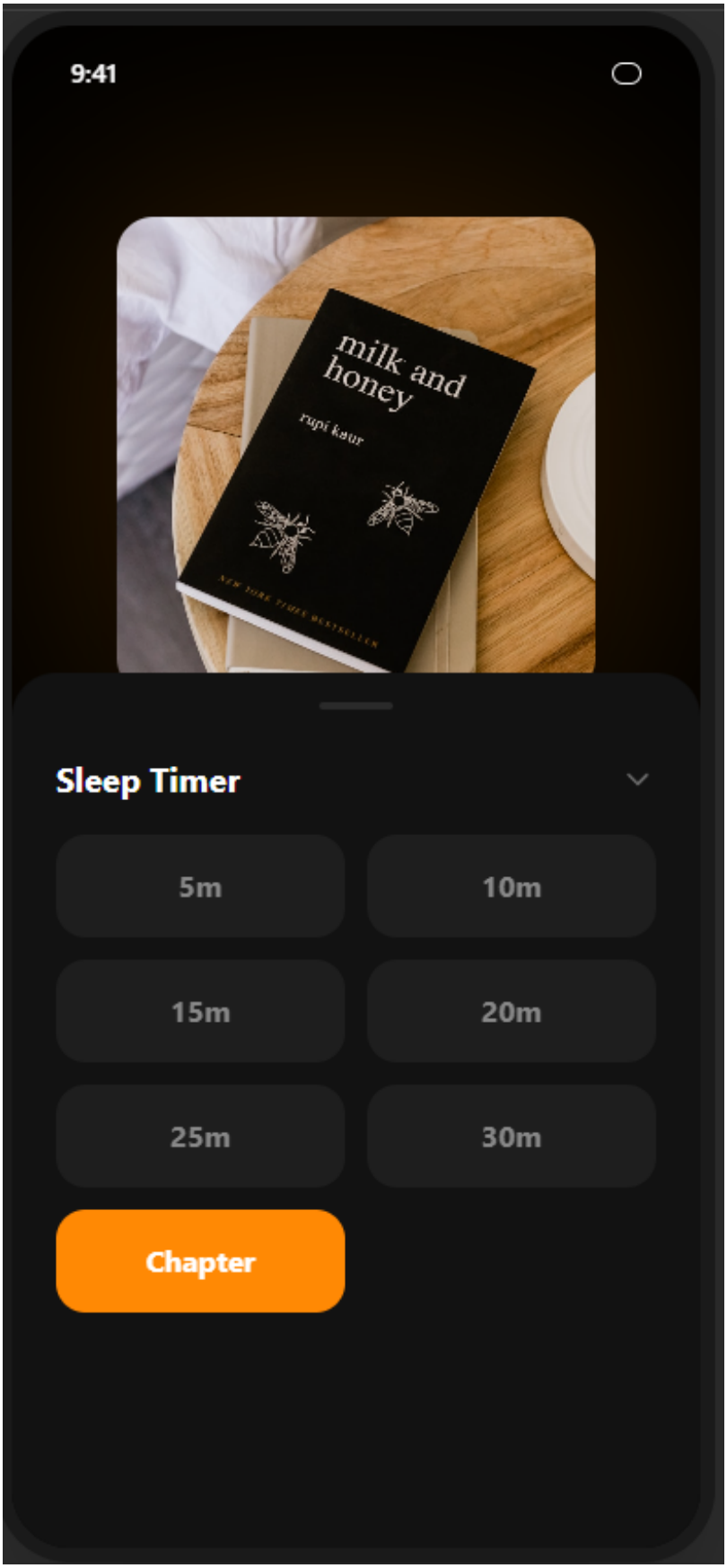
- Optional interactions (chapter/speed/sleep):



Chapter Popup

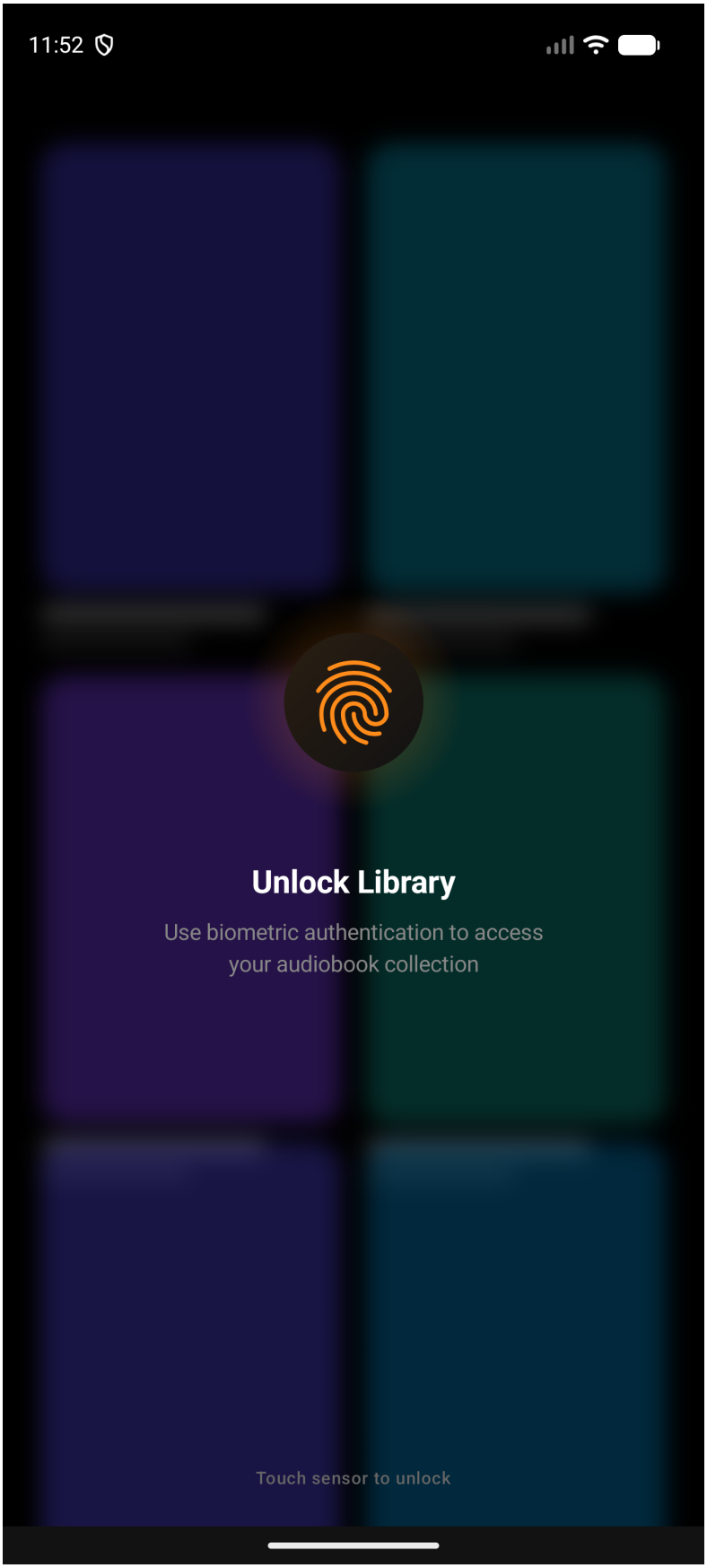

9:41
milk and honey
rupi kaur
The Creative Act
Playback Speed
0.5x
0.75x
1x
1.25x
1.5x
1.75x
2x

Speed Popup

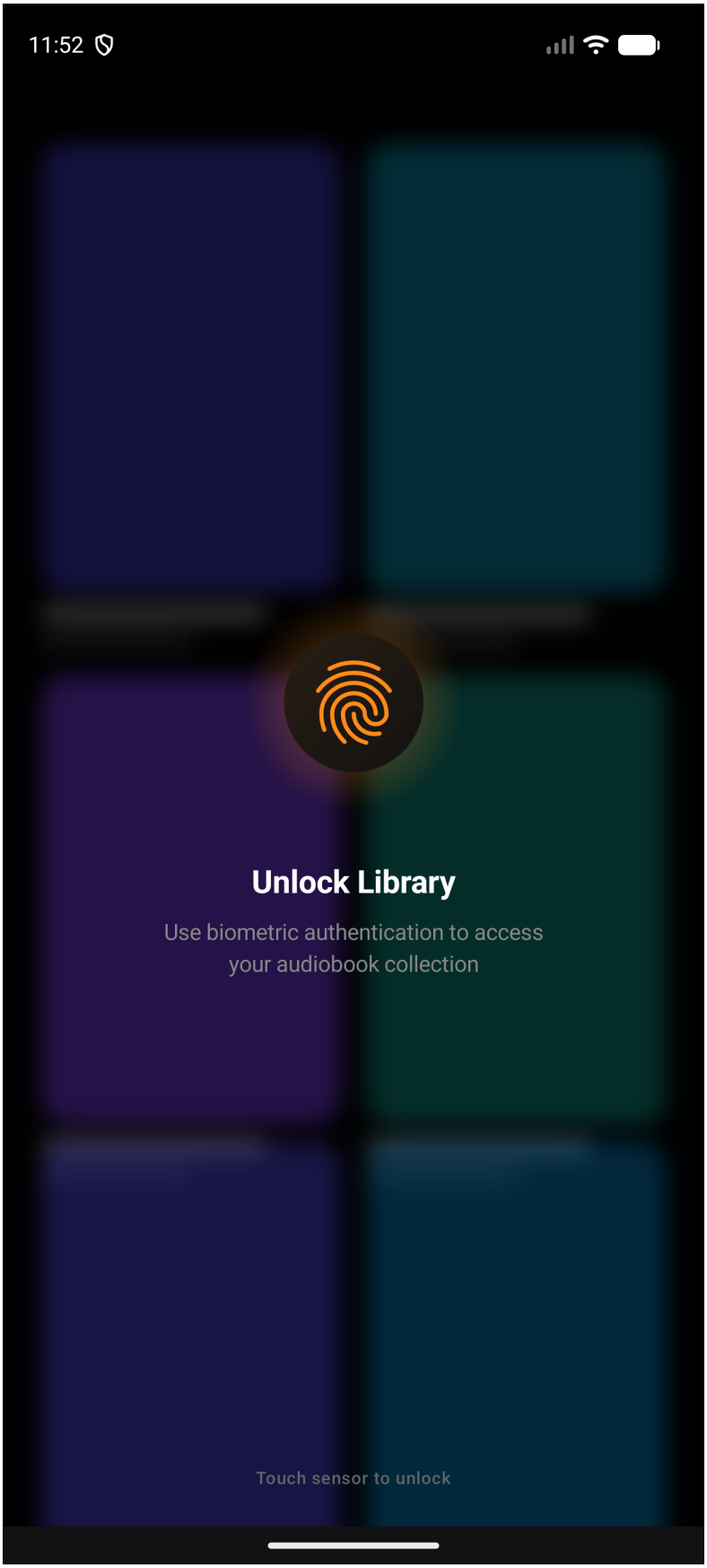


Sleep Timer

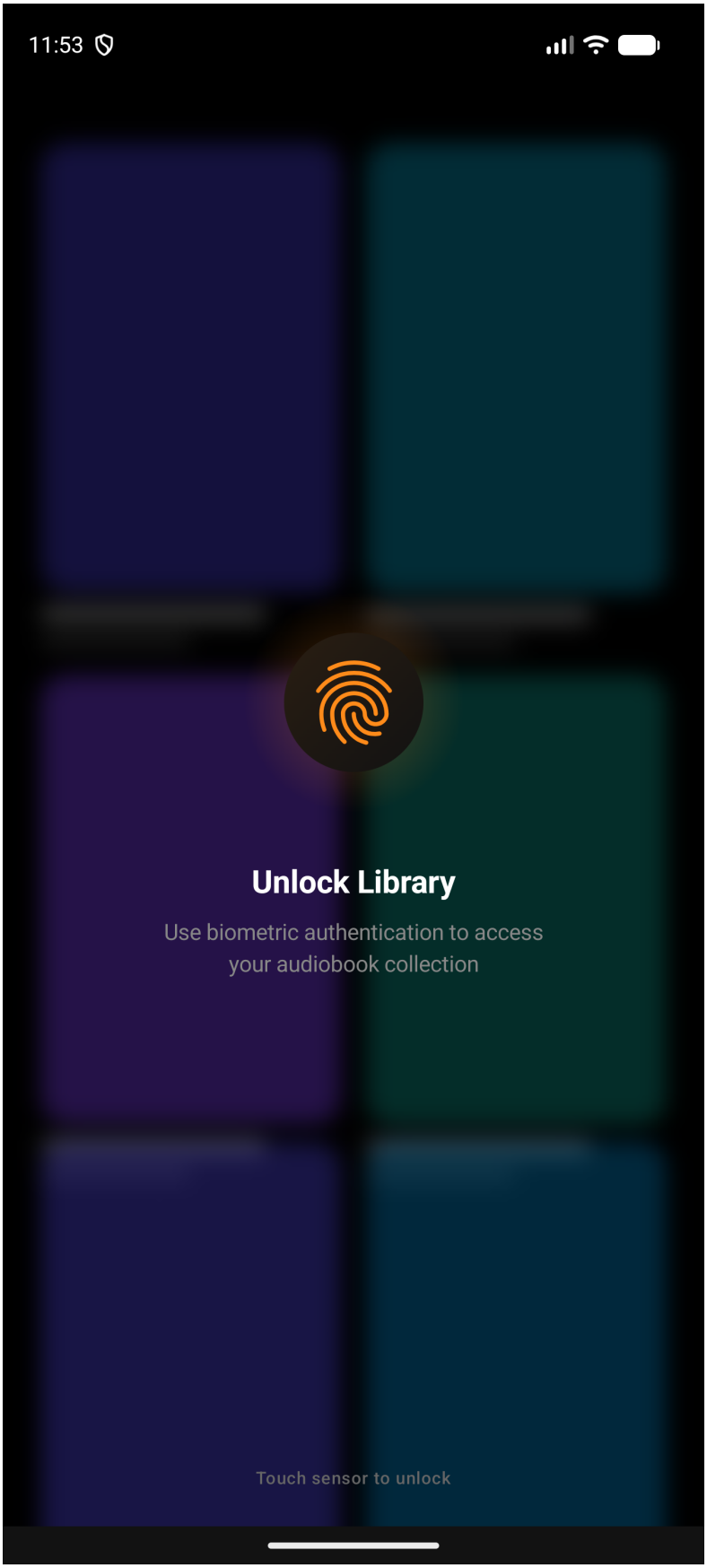
- Emulator snapshots:


11:52
Unlock Library
Use biometric authentication to access your audiobook collection
Touch sensor to unlock

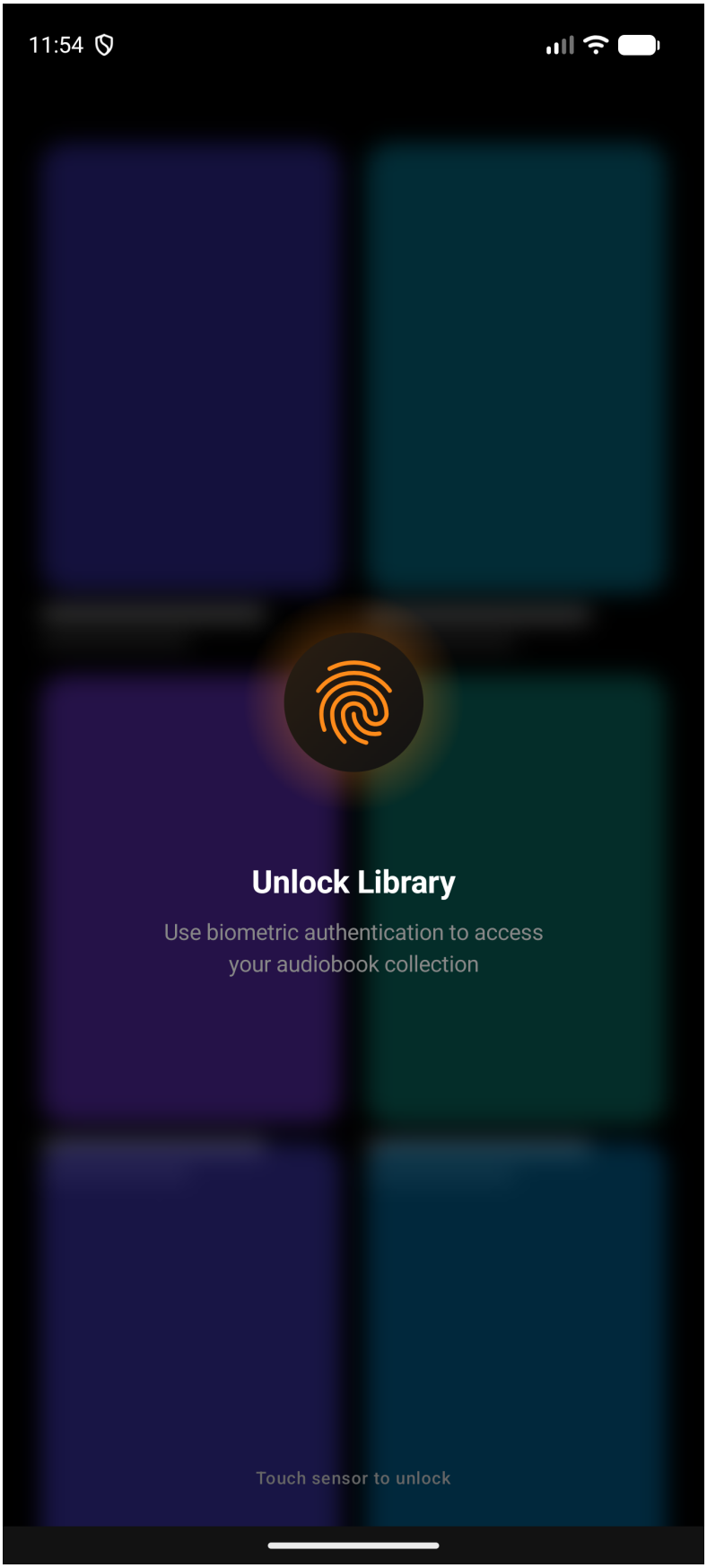
Emulator 1


11:52
Unlock Library
Use biometric authentication to access your audiobook collection
Touch sensor to unlock

Emulator 2


11:53
Unlock Library
Use biometric authentication to access your audiobook collection
Touch sensor to unlock

Emulator 3



Emulator 4

UI flow diagram

Component diagram

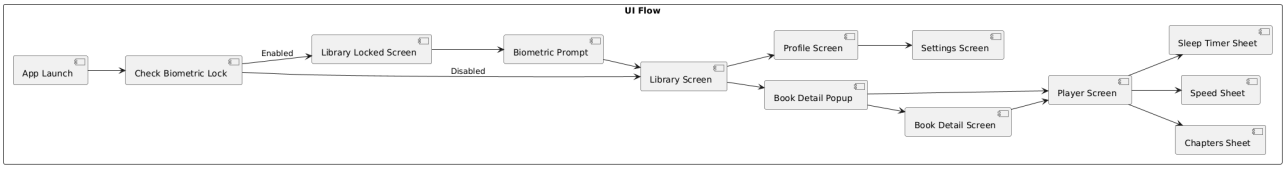
Use case diagram (selected non-trivial use cases)

Exported diagram outputs (PUML)

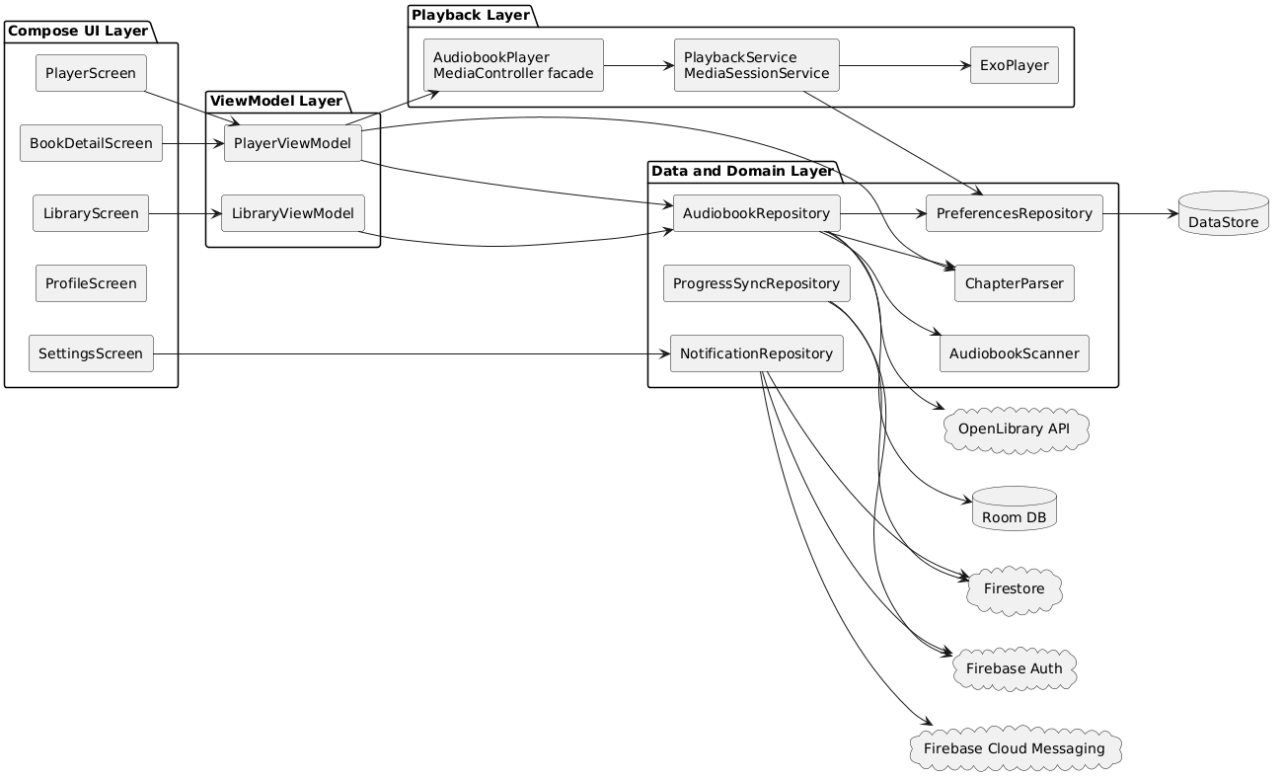
After running the export script, include these rendered outputs in the final PDF package:

- `diagrams/exports/component_diagram.png`

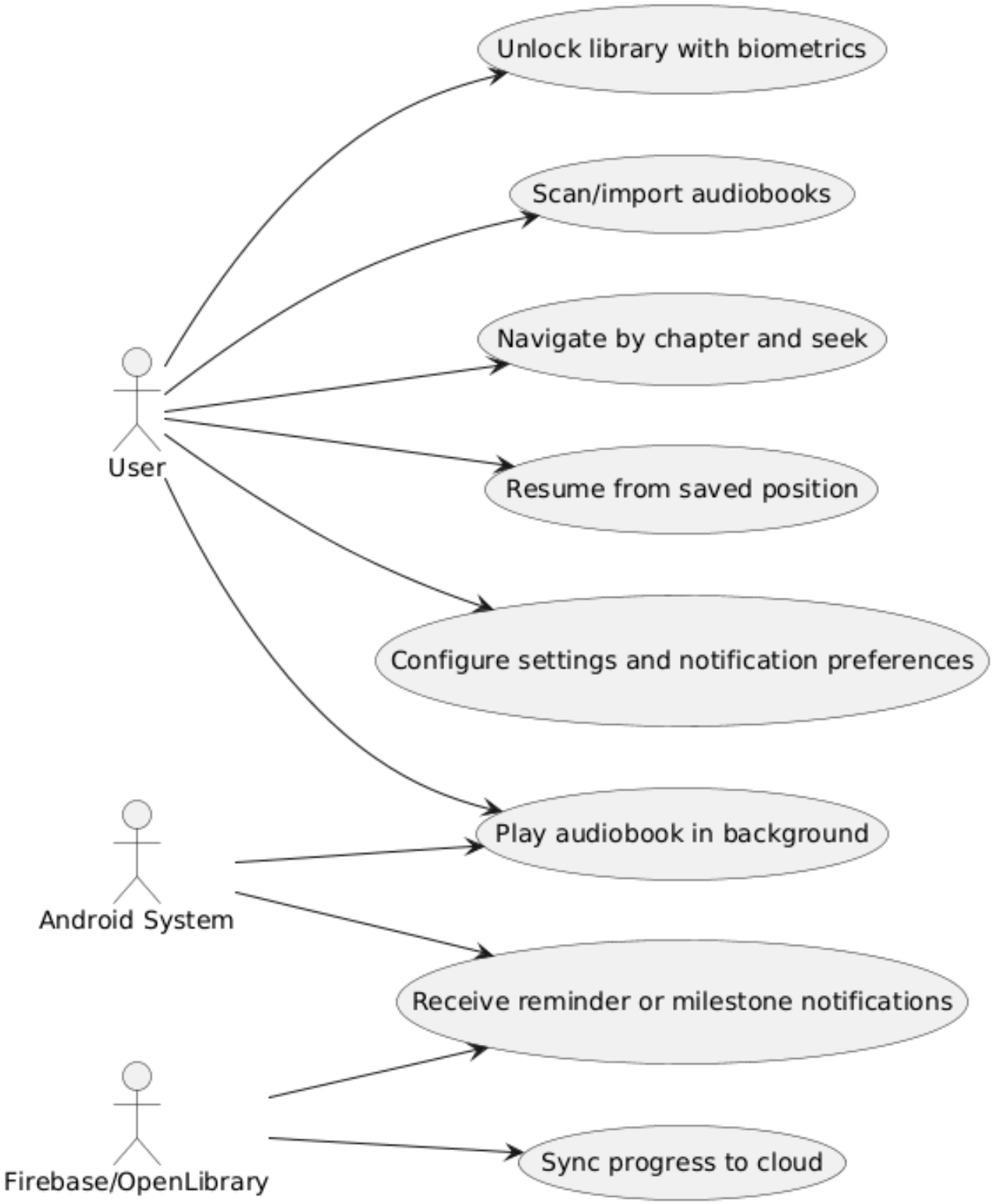
Rendered outputs:



UI Flow Export



Component Diagram Export



Use Case Diagram Export

Conclusion

Lunar Audio delivers a strong end-to-end Android audiobook experience that aligns with the course goals and the original synopsis direction. The project demonstrates practical use of modern Android architecture and media APIs, especially through the Media3 MediaSessionService approach, chapter-aware playback logic, and a Compose-first UI implementation.

From a technical perspective, the main success is the playback architecture: decoupling UI from media

service, preserving playback continuity, and supporting audiobook-specific controls such as 15s/30s skip, chapter navigation, speed control, and sleep timer. The implementation also shows good persistence strategy through Room and DataStore, enabling reliable local resume behavior.

Firebase and notification capabilities are present and substantial, and the project includes advanced ideas such as engagement tracking and milestone-based reminders. In addition, metadata and chapter extraction challenges were investigated deeply, and chapter compatibility improvements were documented, including fallback parsing strategies and generation-side chapter metadata fixes.

The most important insights gained during development were:

- Med

- Long-form audio apps need frequent but controlled persistence updates to balance accuracy and performance.

- Clou

- UX security features (such as biometrics) require complete settings integration, not just prompt-level implementation.

Overall, the project is technically mature in core playback and UI/UX foundations and provides a solid baseline for future production hardening.

Known Bugs and Issues

```
| ID | Issue | Impact | Current Evidence | Suggested Next Step |
|---|
| K1 | Dedicated sign-in/sign-up UI flow is not fully wired in navigation | Users cannot complete full account lifecycle in-app from a clear auth screen flow | Auth repository exists, but navigation does not define auth screens | Add explicit auth screens and route guards tied to auth state |
| K2
| K3 | Biometric fallback bypass remains in demo path | On unsupported devices, lock can be bypassed, weakening security expectation | `MainActivity` contains "bypass for demo" fallback | Replace bypass with strict policy or explicit PIN/passcode fallback |
| K4
| K5 | Firestore progress sync repository is underused in main flow | Cross-device continuity may be incomplete in real usage | `ProgressSyncRepository` API exists but few/no integration call sites | Integrate save/pull/observe in startup, playback pause, and conflict points |
| K6
| K7 | Notification icons still use placeholder system icon | Inconsistent branding and potentially lower notification quality | TODO comments in scheduler and FCM service | Replace with app-specific monochrome notification icon |
| K8
| K9 | Chapter metadata format compatibility depends on source format | Some M4B files may still need regeneration for reliable chapter behavior | Chapter fix docs note MDTA reliability limits and recommend Nero chapters | Standardize chapter generation pipeline (prefer Nero/mp4chaps) |
```

Final Note

This report is written in the same language as the synopsis (English) and follows the required section order from the project report rubric.